Algorithms

Lecture #22

Syed Hamed Raza

O/1 Knapsack Problem

0/1 Knapsack Problem

- To compute entries of V we will imply an inductive approach.
- As a basis, $V[0, j] = 0$ for $0 \leq j \leq W$ since if we have no items then we have no value.
- We consider two cases:

O/1 Knapsack Problem

0/1 Knapsack Problem

Leave object *i*:

- If we choose to not take object i, then theoptimal value will come about by considering how to fill a knapsack of size j with the remaining objects $\{1, 2, \ldots, i - 1\}$.
- This is just $V[i - 1, j]$.

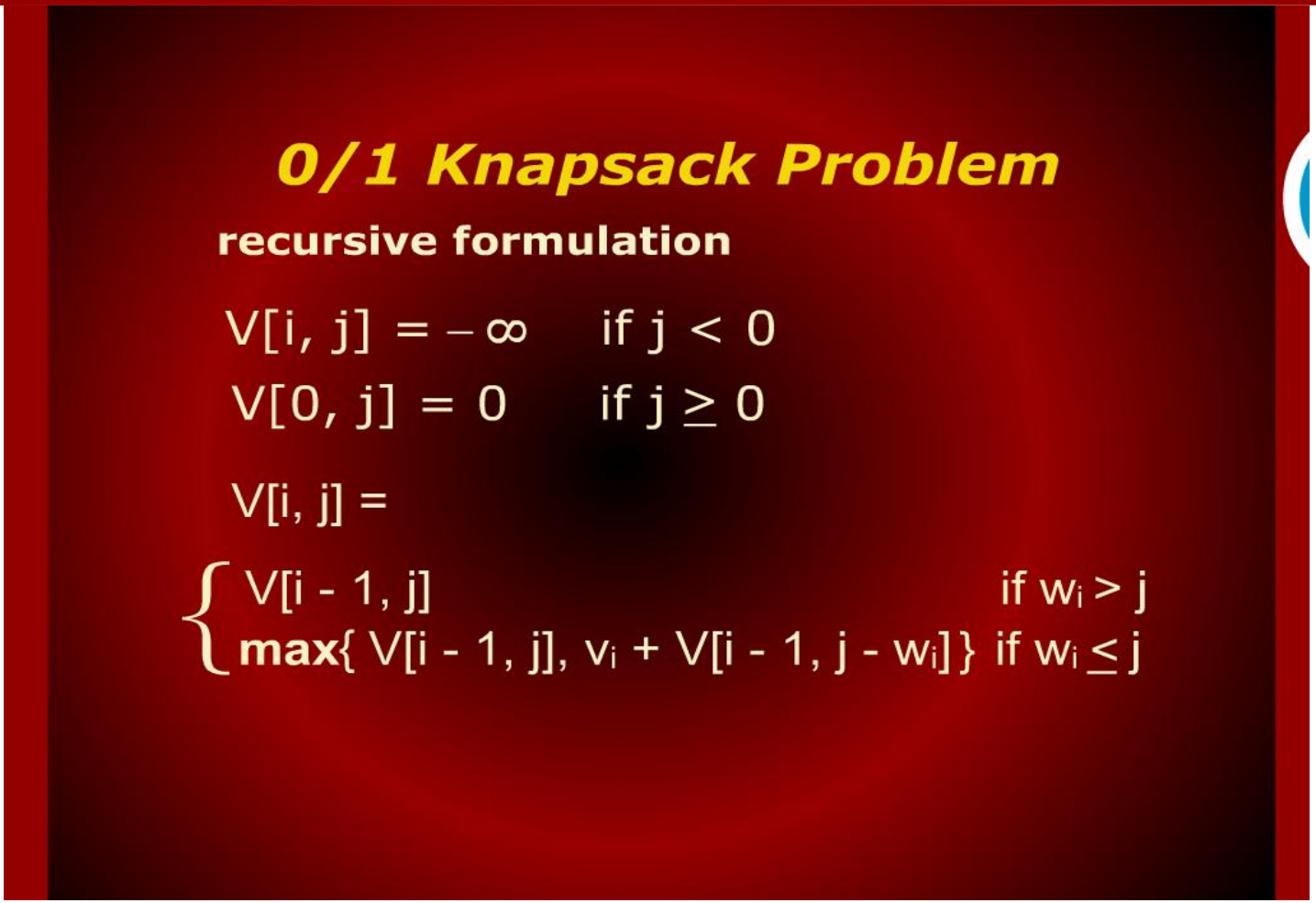
O/1 Knapsack Problem

0/1 Knapsack Problem

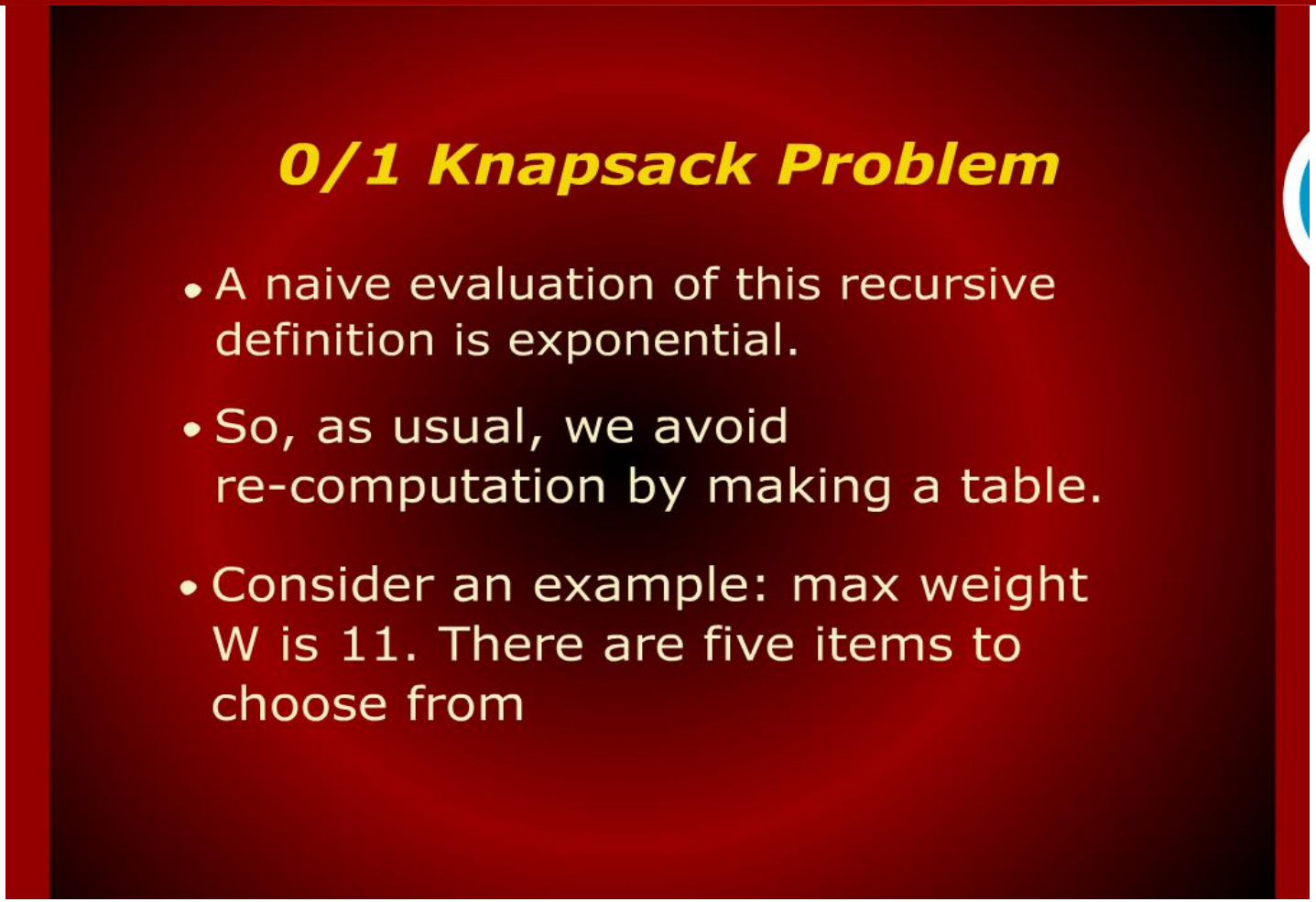
Take object i:

- If we take object i, then we gain a value of v_i
- But we use up w_i of our capacity.
- With the remaining $j - w_i$ capacity in the knapsack, we can fill it in the best possible way with objects $\{1, 2, \ldots, i - 1\}$.
- This is $v_i + V[i - 1, j - w_i]$.
- This is only possible if $w_i \leq j$.

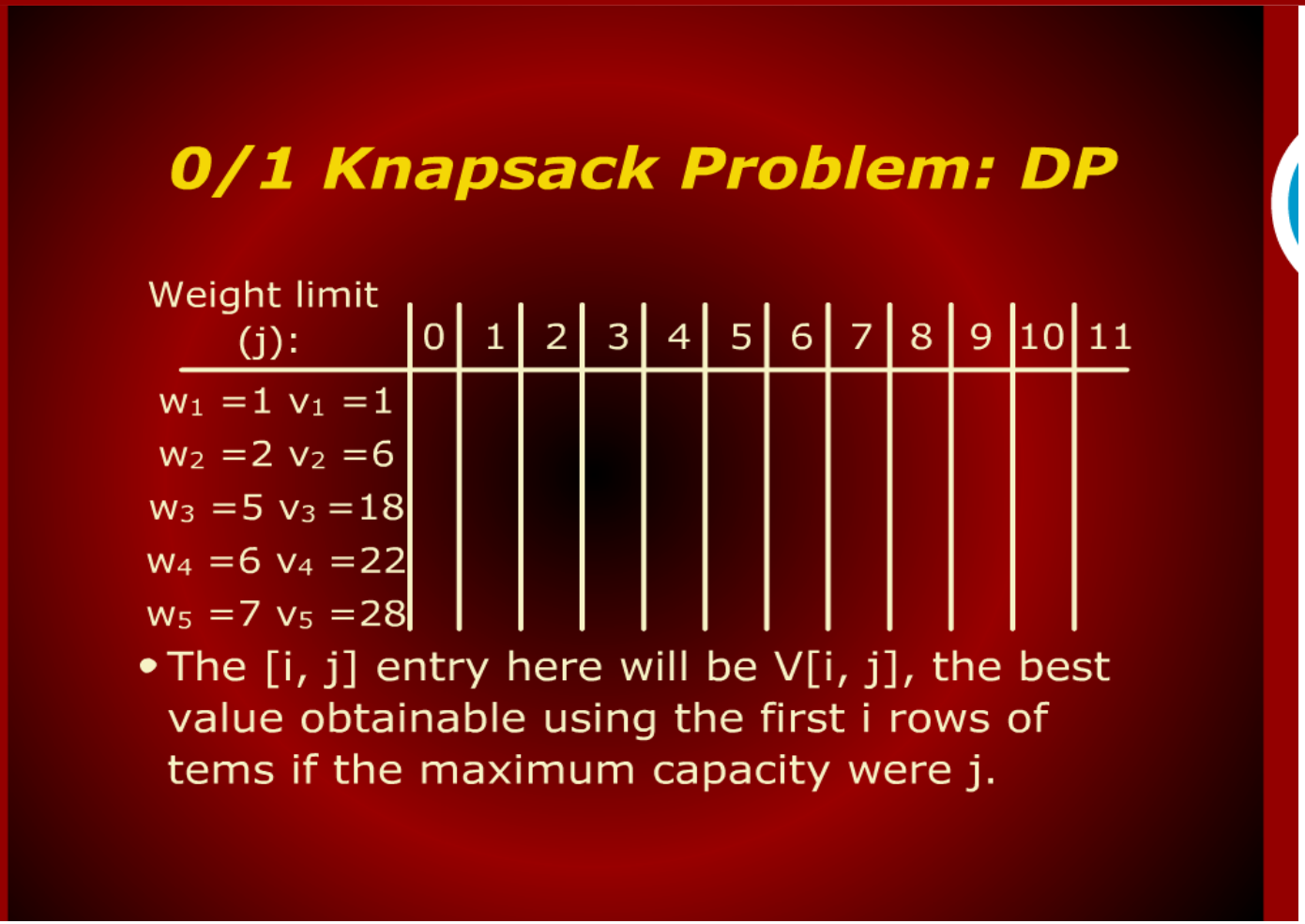
O/1 Knapsack Problem



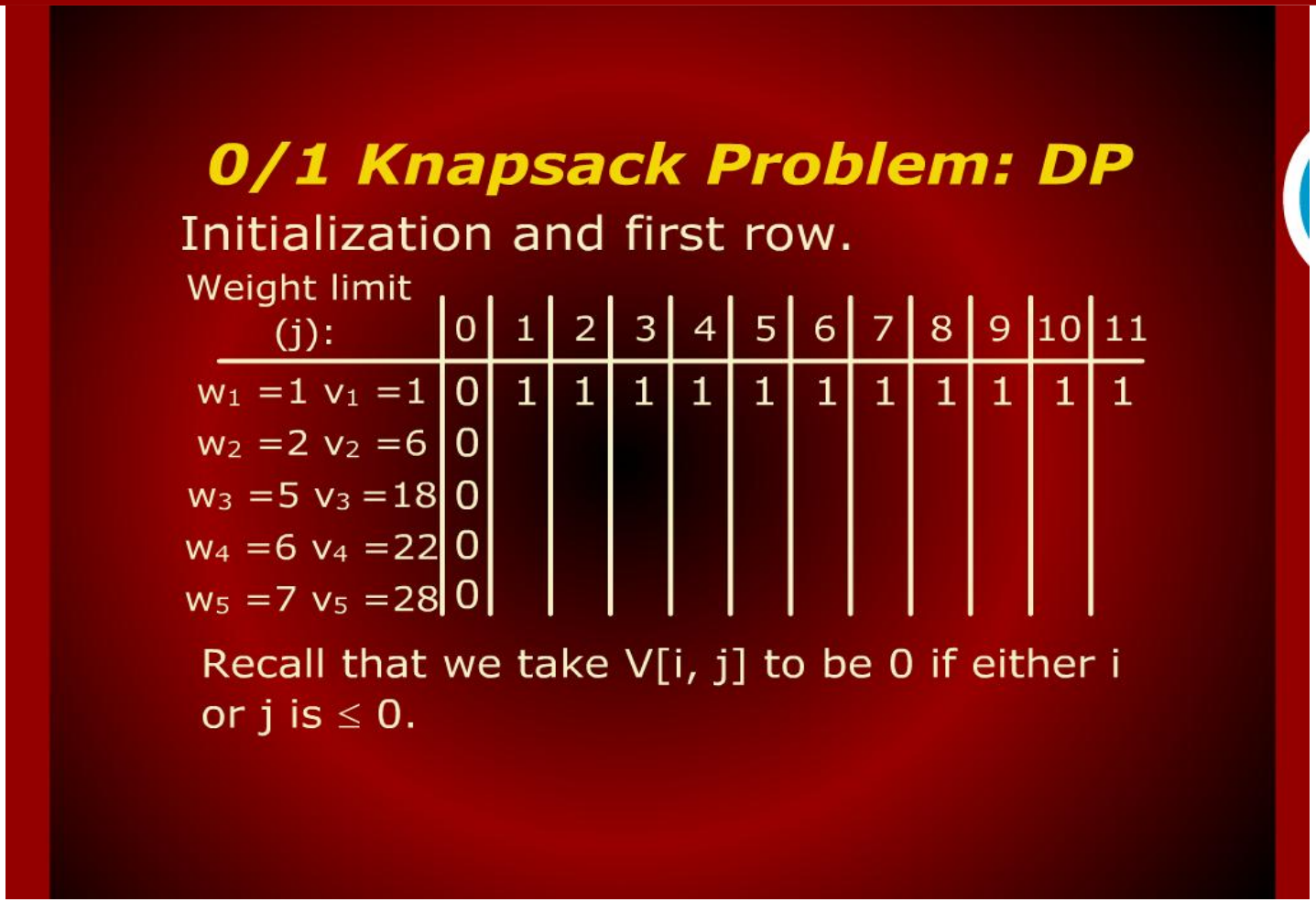
O/1 Knapsack Problem



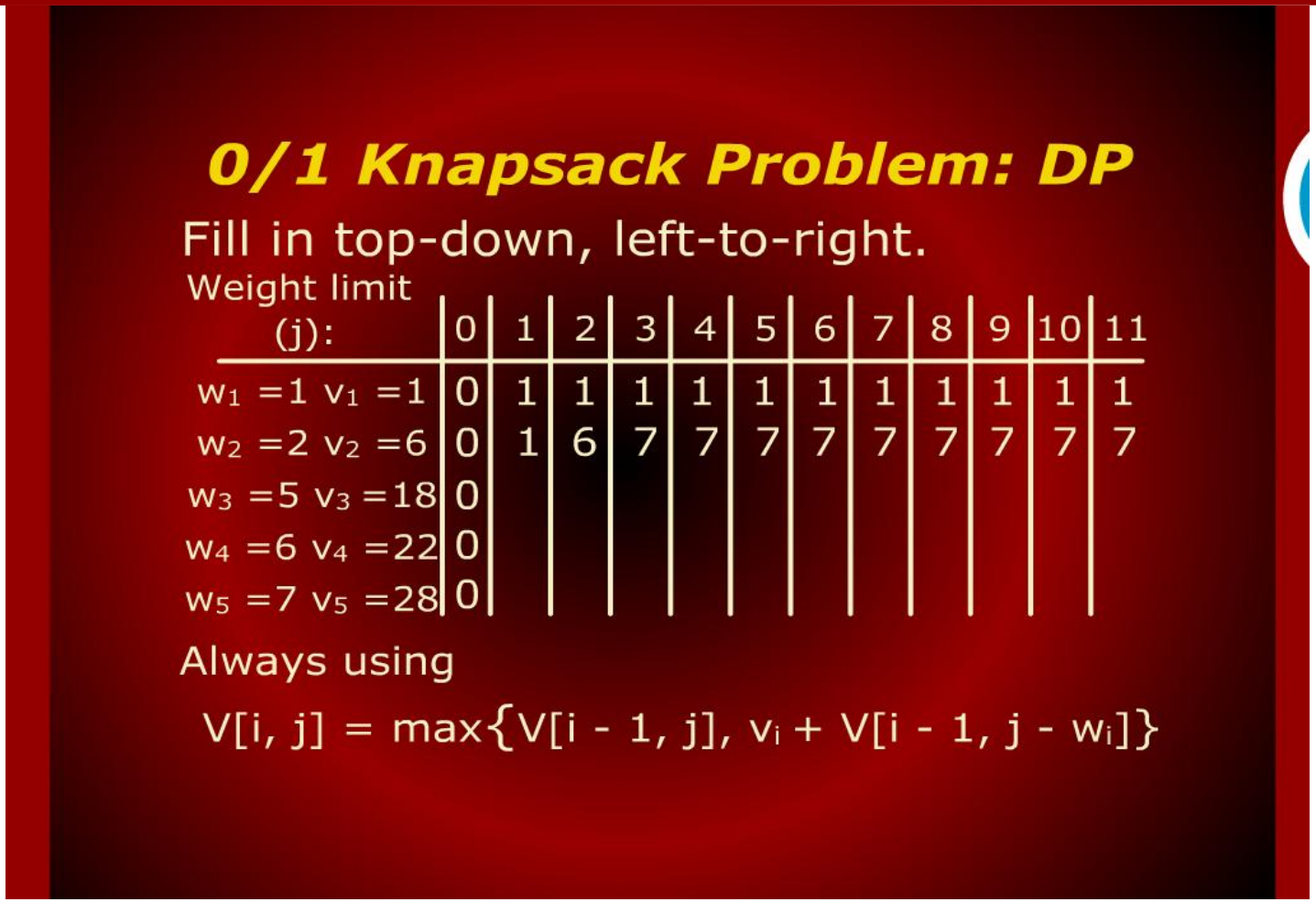
O/1 Knapsack Problem: DP



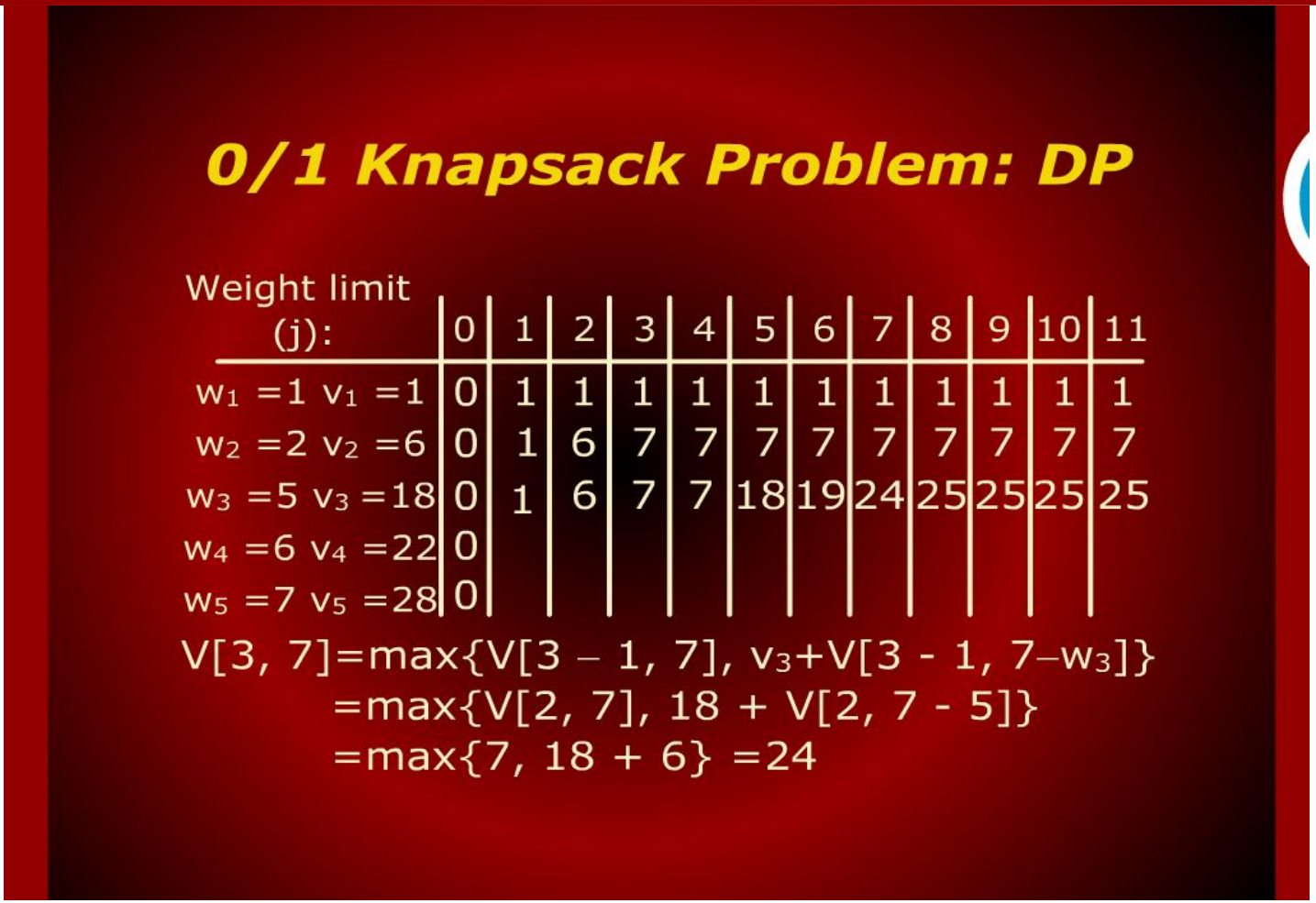
O/1 Knapsack Problem: DP



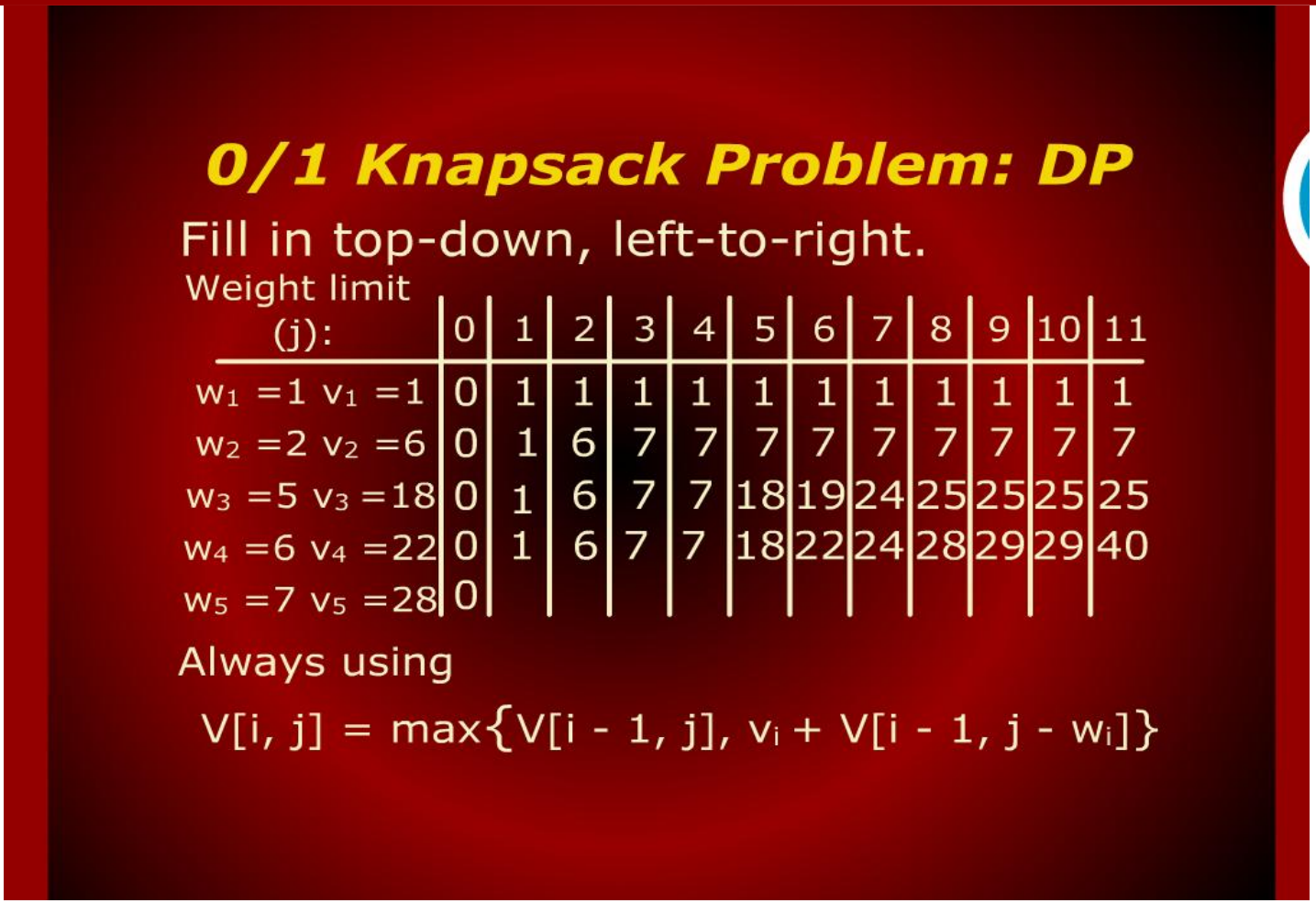
O/1 Knapsack Problem: DP



O/1 Knapsack Problem: DP

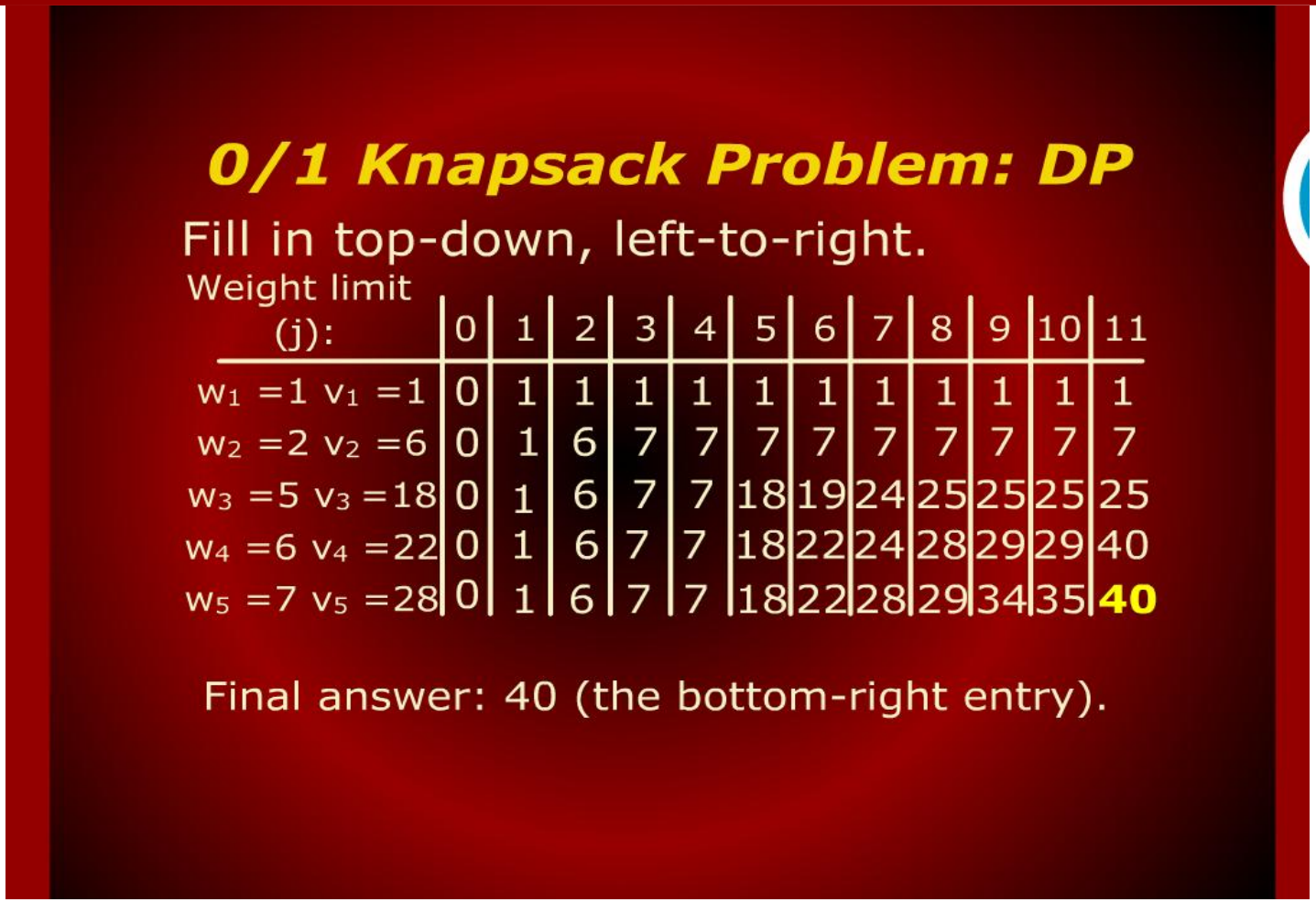


O/1 Knapsack Problem: DP



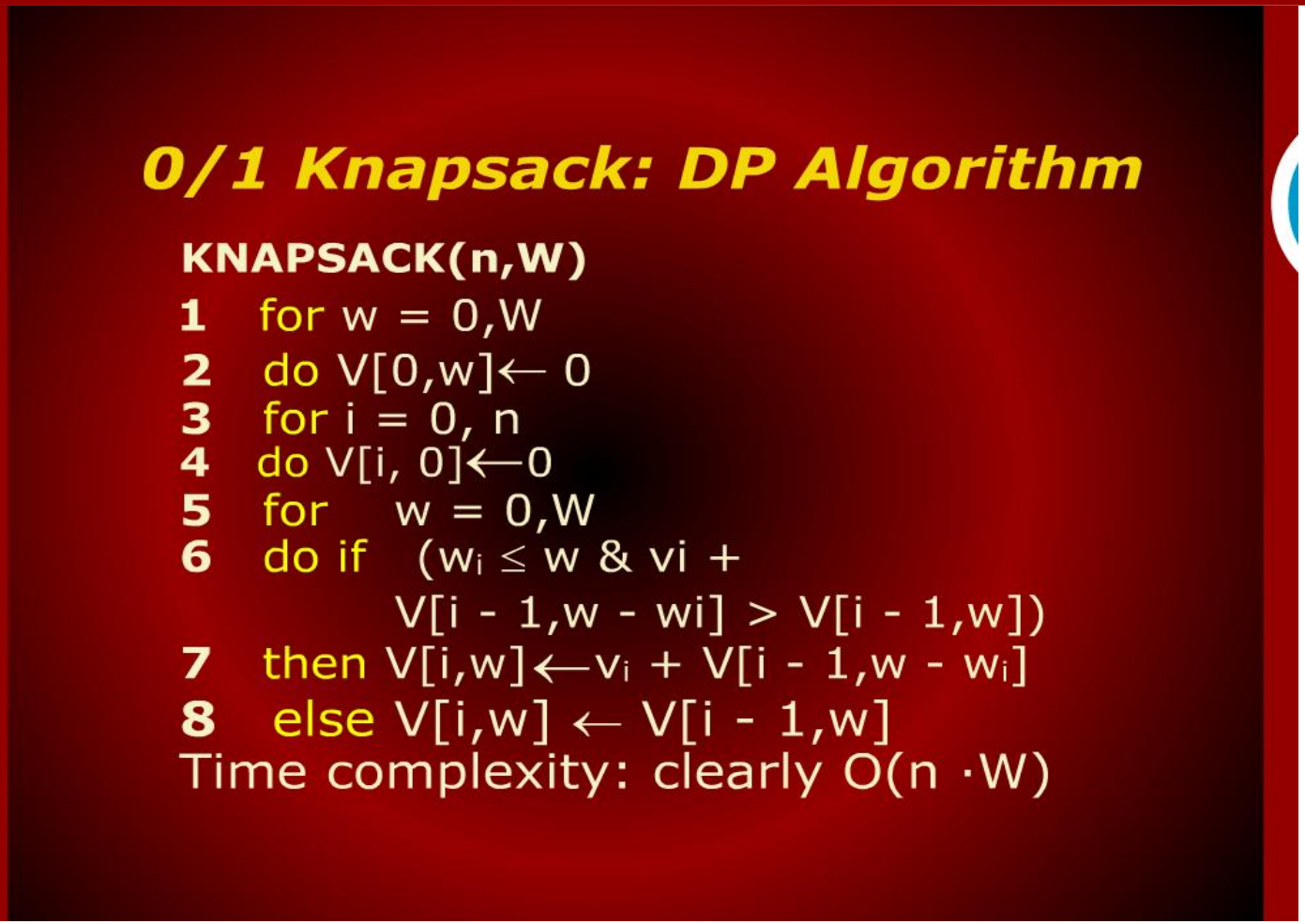
Weight limit (j):	0	1	2	3	4	5	6	7	8	9	10	11
w_1 =1 v_1 =1	0	1	1	1	1	1	1	1	1	1	1	1
w_2 =2 v_2 =6	0	1	6	7	7	7	7	7	7	7	7	7
w_3 =5 v_3 =18	0	1	6	7	7	18	19	24	25	25	25	25
w_4 =6 v_4 =22	0	1	6	7	7	18	22	24	28	29	29	40
w_5 =7 v_5 =28	0											

O/1 Knapsack Problem: DP



Weight limit (j):	0	1	2	3	4	5	6	7	8	9	10	11
w_1 =1 v_1 =1	0	1	1	1	1	1	1	1	1	1	1	1
w_2 =2 v_2 =6	0	1	6	7	7	7	7	7	7	7	7	7
w_3 =5 v_3 =18	0	1	6	7	7	18	19	24	25	25	25	25
w_4 =6 v_4 =22	0	1	6	7	7	18	22	24	28	29	29	40
w_5 =7 v_5 =28	0	1	6	7	7	18	22	28	29	34	35	**40**

O/1 Knapsack: DP Algorithm



O/1 Knapsack Problem: DP

0/1 Knapsack Problem: DP

Constructing the Optimal Solution

- The algorithm for computing V[i, j] does not keep record of which subset of items gives the optimal solution.
- To compute the actual subset, we can add an auxiliary boolean array keep[i, j] which is 1 if we decide to take the ith item and 0 otherwise.

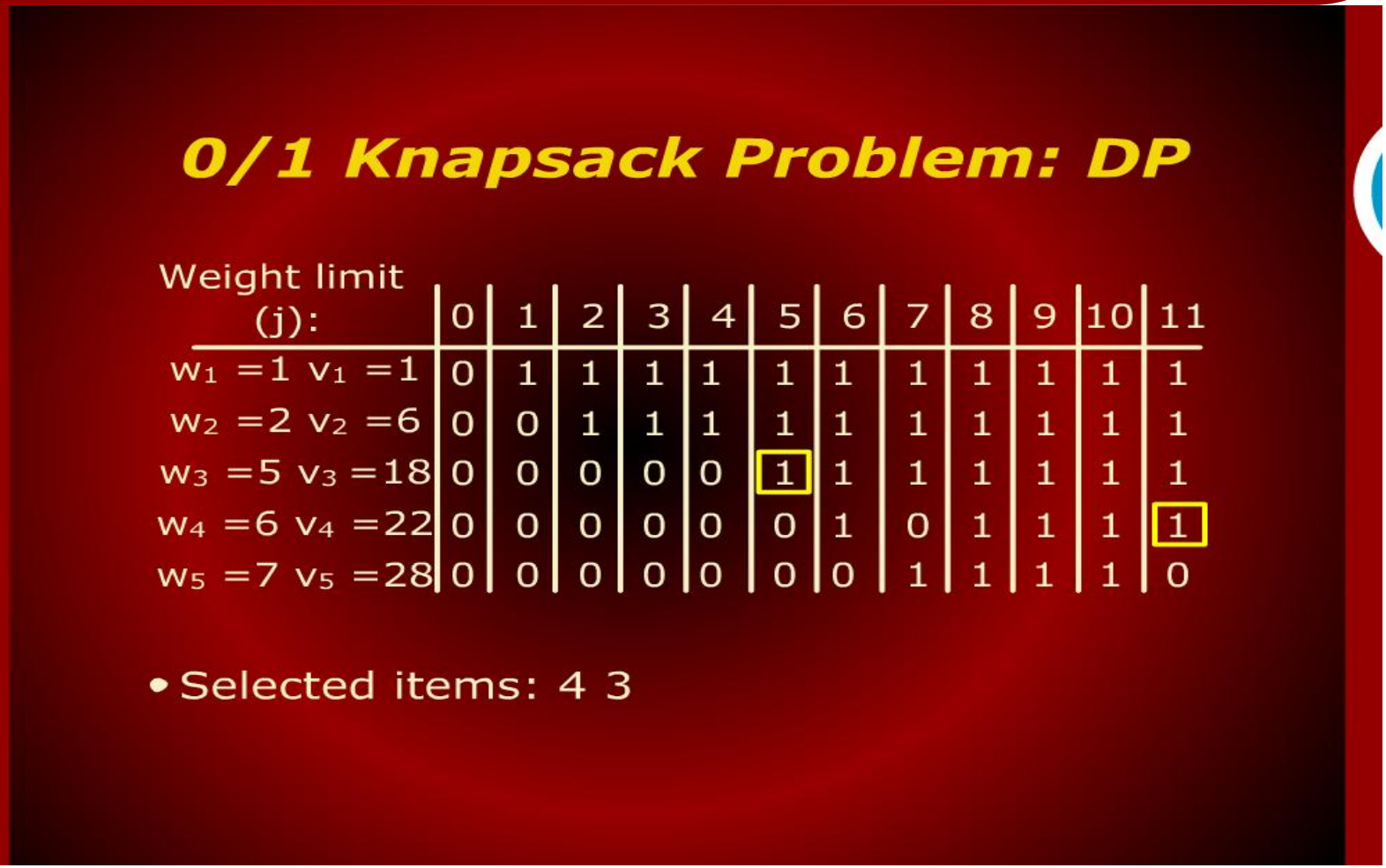
O/1 Knapsack Problem

0/1 Knapsack Problem

Question: How do we use all the values keep[i, j] to determine the optimal subset T of items?

- If keep[n,W] is 1, then $n \in T$. We can now repeat this argument for keep[n - 1,W - w_n].
- If kee[n,W] is 0, the $n \notin T$ and we repeat the argument for keep [n - 1,W].

O/1 Knapsack Problem: DP



Design and Analysis of Algorithms

O/1
Knapsack

O/1 Knapsack Problem

0/1 Knapsack Problem

```
KNAPSACK(n,W)
1   for w = 0,W
2   do V[0,w] ← 0
3   for i = 0, n
4   do V[i, 0] ← 0
5       for w = 0,W
6       do if (wi ≤ w & vi +
              V[i - 1,w - wi]> V[i - 1,w])
```

O/1 Knapsack Problem

0/1 Knapsack Problem

Therefore, the following partial program will output the elements of T: . . . ()

```
1   k ← W
2   for i ← n downto 1
3   do if (keep[i, k] = 1)
4         then output i
5         k ← k - w_i
```